



HCMC UNIVERSITY OF TECHNOLOGY
FACULTY OF CSE

MOCK TEST

Sem - A.year	2	2024-2025
Date	17/5/2025	

Course	Advanced Programming
Course ID	CO2039
Time	90 minutes

Question 1 (L.O.1). What is a characteristic of state-less programming in functional programming?

- ☐ A Functions modify external state.
- ☐ B Functions produce the same output for the same input.
- ☐ C Functions rely on global variables.
- ☐ D Functions have side effects.

Question 2 (L.O.2). What is the output of the following Java code?

```
class Vehicle {
    String type = "Vehicle";
    void display() { System.out.println(type); }
}
class Car extends Vehicle {
    String type = "Car";
}
public class Main {
    public static void main(String[] args) {
        Vehicle v = new Car();
        v.display();
    }
}
```

- ☐ A Compilation error.
- ☐ B Car.
- ☐ C Vehicle.
- ☐ D Null.

Question 3 (L.O.1). In Python, what does the map higher-order function do?

- ☐ A Filters elements based on a condition.
- ☐ B Combines elements into a single value.
- ☐ C Applies a function to each element in an iterable.
- ☐ D Creates a new function.

Question 4 (L.O.1). In Java, complete the following Singleton pattern code by filling in the blank:

```
public class Singleton {
```

```
    private static Singleton instance = null;
    ;
    private Singleton() {}
    public static Singleton getInstance() {
        -----
        return instance;
    }
}
```

- ☐ A if (instance != null) { instance = new Singleton(); }
- ☐ B return new Singleton();
- ☐ C instance = new Singleton();
- ☐ D if (instance == null) { instance = new Singleton(); }

Question 5 (L.O.3). What is the output of the following Python code?

```
f = lambda x: lambda y: x * y
g = f(3)
print(g(4))
```

- ☐ A 12.
- ☐ B 7.
- ☐ C 4.
- ☐ D None.

Question 6 (L.O.1). In Java, what is a key difference between an abstract class and an interface?

- ☐ A An interface can have concrete methods, but an abstract class cannot.
- ☐ B An abstract class cannot be inherited, but an interface can.
- ☐ C An abstract class can have instance variables, but an interface cannot.
- ☐ D An interface can have private methods, but an abstract class cannot.

Question 7 (L.O.1). In C++, how is runtime polymorphism achieved?

- ☐ A By using static methods.
- ☐ B By making methods private.

- Ⓒ By declaring methods as virtual in the base class.
- Ⓓ By using inline functions.

Question 8 (L.O.3). Complete the following Python code to filter even numbers:

```
numbers = [1, 2, 3, 4, 5]
result = list(_____)
print(result)
```

- Ⓐ filter(lambda x: x % 2 == 0, numbers).
- Ⓑ map(lambda x: x % 2 == 0, numbers).
- Ⓒ filter(lambda x: x % 2 != 0, numbers).
- Ⓓ map(lambda x: x * 2, numbers).

Question 9 (L.O.2). In *Assignment 1*, what is the output of the following code for a DeathEater with mana having real part 3 and imaginary part 4?

```
public class DeathEater extends Monster
    implements Combatable {
    private Complex mana;
    public DeathEater(Complex mana) {
        this.mana = mana;
    }
    @Override
    public double getCombatScore() {
        return mana.getMagnitude();
    }
}
```

Assume `Complex::getMagnitude()` returns $\sqrt{\text{real}^2 + \text{imaginary}^2}$.

- Ⓐ 12.0.
- Ⓑ 5.0.
- Ⓒ 7.0.
- Ⓓ 3.0.

Question 10 (L.O.1). Which of the following is a built-in higher-order function in Python?

- Ⓐ len.
- Ⓑ print.
- Ⓒ filter.
- Ⓓ int.

Question 11 (L.O.1). In Java, what happens if a derived class constructor does not explicitly call the base class constructor?

- Ⓐ A runtime error occurs.
- Ⓑ The base class default constructor is automatically called.
- Ⓒ The program will not compile.
- Ⓓ The base class constructor is skipped.

Question 12 (L.O.2). In *Assignment 2*, complete the following Java code to implement the Adapter pattern:

```
package com.myproject;
import java.util.List;
import java.util.stream.Collectors;
public class HoseAdapter implements
    PriceFetcher {
    private HosePriceFetchLib hoseLib;
    private List<String> stockCodes;
    public HoseAdapter(HosePriceFetchLib
        hoseLib, List<String> stockCodes) {
        this.hoseLib = hoseLib;
        this.stockCodes = stockCodes;
    }
    @Override
    public List<StockPrice> fetch() {
        -----
    }
    private StockPrice convertToStockPrice(
        HoseData hoseData) {
        return new StockPrice(hoseData.
            getStock(), hoseData.getPrice(),
            hoseData.getTime(), hoseData.
            getVolume());
    }
}
```

- Ⓐ return new ArrayList();
- Ⓑ return hoseLib.getStockData(stockCodes);
- Ⓒ return hoseLib.getStockData(stockCodes).stream().map(this::convertToStockPrice).collect(Collectors.toList());
- Ⓓ return stockCodes.stream().map(hoseLib::getStockData).collect(Collectors.toList());

Question 13 (L.O.3). What is the output of the following Python code?

```
def square(x):
    return x * x
numbers = [1, 2, 3]
result = list(map(square, numbers))
print(result)
```

- Ⓐ [].
- Ⓑ [2, 4, 6].
- Ⓒ [1, 2, 3].
- Ⓓ [1, 4, 9].

Question 14 (L.O.1). In Java, which design pattern ensures a class has only one instance?

- (A) Observer.
- (B) Adapter.
- (C) Factory.
- (D) Singleton.

Question 15 (L.O.1). In Python, what does the following list comprehension produce?

```
numbers = [1, 2, 3, 4]
result = [x + 1 for x in numbers]
```

- (A) [2, 3, 4, 5].
- (B) [1, 2, 3, 4].
- (C) [1, 3, 5, 7].
- (D) [0, 1, 2, 3].

Question 16 (L.O.2). Complete the following Python code to correctly implement inheritance:

```
class Vehicle:
    def __init__(self, model):
        self.model = model
    def info(self):
        return f"Model: {self.model}"
class Car(Vehicle):
    def __init__(self, model):
        -----
car = Car("Toyota")
print(car.info())
```

- (A) self.model = model
- (B) Vehicle.init(model)
- (C) super().init(model)
- (D) self.info = model

Question 17 (L.O.1). In a UML class diagram for *Assignment 2*, what is the relationship between StockFeeder and StockViewer?

```
interface StockViewer {
    void onUpdate(StockPrice stockPrice);
}
class StockFeeder {
    private Map<String, List<StockViewer>>
    viewers = new HashMap<>();
    public void registerViewer(String code,
        StockViewer viewer) {
        viewers.computeIfAbsent(code, k ->
            new ArrayList<>()).add(viewer);
    }
}
```

- (A) Composition.
- (B) Dependency.

- (C) Inheritance.
- (D) Association.

Question 18 (L.O.1). What is a key characteristic of a pure function in Python?

- (A) It modifies external state.
- (B) It uses global variables.
- (C) It produces no side effects.
- (D) It has no return value.

Question 19 (L.O.1). In Java, what is the primary purpose of the Adapter pattern?

- (A) To notify observers of state changes.
- (B) To ensure a class has only one instance.
- (C) To allow incompatible interfaces to work together.
- (D) To encapsulate object creation.

Question 20 (L.O.3). Complete the following Python code to double numbers using a lambda function:

```
numbers = [1, 2, 3, 4]
doubled = list(-----)
print(doubled)
```

- (A) filter(lambda x: x * 2, numbers)
- (B) map(lambda x: x * 2, numbers)
- (C) filter(lambda x: x > 0, numbers)
- (D) map(lambda x: x + 2, numbers)

Question 21 (L.O.2). What is the output of the following C++ code?

```
#include <iostream>
class Animal {
public:
    virtual void sound() { std::cout << "
        Generic\n"; }
};
class Dog : public Animal {
public:
    void sound() override { std::cout << "
        Woof\n"; }
};
int main() {
    Animal* animal = new Dog();
    animal->sound();
    delete animal;
    return 0;
}
```

```
}
```

- ☐ A Generic.
- ☐ B Woof.
- ☐ C No output.
- ☐ D Compilation error.

Question 22 (L.O.1). In a UML class diagram, what does a solid line with a closed arrowhead represent?

- ☐ A Association.
- ☐ B Dependency.
- ☐ C Inheritance.
- ☐ D Aggregation.

Question 23 (L.O.1). In functional programming, what does stateless programming mean?

- ☐ A Functions produce the same output for the same input and have no side effects.
- ☐ B Functions cannot return values.
- ☐ C Functions store data in global variables.
- ☐ D Functions must use loops.

Question 24 (L.O.2). In *Assignment 1*, complete the following Java code to calculate the combat score:

```
public class Warrior extends Fighter {
    private int baseHp;
    private int wp;
    public Warrior(int baseHp, int wp) {
        this.baseHp = baseHp;
        this.wp = wp;
    }
    @Override
    public double getCombatScore() {
        -----
    }
}
```

- ☐ A return wp == 1 ? baseHp : baseHp / 10.0;
- ☐ B return baseHp + wp;
- ☐ C return baseHp * wp;
- ☐ D return baseHp / wp;

Question 25 (L.O.3). What is the output of the following Python code?

```
numbers = [-2, -1, 0, 1, 2]
result = [x * 2 for x in numbers if x < 0]
print(result)
```

- ☐ A [-4, -2].
- ☐ B [-2, -1].
- ☐ C [].
- ☐ D [4, 2, 0, 2, 4].

Question 26 (L.O.2). In *Assignment 2*, what is the purpose of the PriceFetcher interface?

- ☐ A To notify viewers of price updates.
- ☐ B To manage stock codes.
- ☐ C To fetch stock prices from external sources.
- ☐ D To log errors to the console.

Question 27 (L.O.1). In Python, how is information hiding typically achieved?

- ☐ A By using double underscores (__) to mangle attribute names.
- ☐ B By declaring attributes as private.
- ☐ C By using the protected keyword.
- ☐ D By making all attributes public.

Question 28 (L.O.3). In *Assignment 3*, complete the following Python code to handle the LOOKUP command:

```
from StaticError import *
from Symbol import *
def simulate(list_of_commands):
    symbols = []
    def insert(name, typ):
        if any(s.name == name for s in symbols):
            raise Redeclared(f"INSERT {name} {typ}")
        symbols.append(Symbol(name, typ))
        return "success"
    def lookup(name, index=len(symbols) - 1):
        :
        -----
    results = []
    for cmd in list_of_commands:
        parts = cmd.split()
        if parts[0] == "INSERT":
            results.append(insert(parts[1], parts[2]))
        elif parts[0] == "LOOKUP":
            results.append(lookup(parts[1]))
    return results
```

- ☐ A return "success"

- ```

B if index < 0:
 □ raise Undeclared(f"LOOKUP name")
 elif symbols[index].name == name:
 □ return index
 else:
 □ return lookup(name, index - 1)

C raise Redeclared(f"LOOKUP name")

D return symbols[0].name

```

**Question 29 (L.O.1).** In Java, what is the output of the following Adapter pattern implementation?

```

interface Target {
 String request();
}

class Adaptee {
 String specificRequest() { return "
 Adaptee"; }
}

class Adapter implements Target {
 private Adaptee adaptee;
 public Adapter(Adaptee adaptee) {
 this.adaptee = adaptee;
 }
 public String request() {
 return adaptee.specificRequest() + "
 via Adapter";
 }
}

public class Main {
 public static void main(String[] args) {
 Adaptee adaptee = new Adaptee();
 Target target = new Adapter(adaptee);
 System.out.println(target.request());
 }
}

```

- ☐ A Compilation error     ☐ B Adaptee  
☐ C Adaptee via Adapter     ☐ D Runtime error

**Question 30 (L.O.1).** In functional programming, what is currying?

- ☒ A Combining multiple functions into one.
- ☐ B Creating a function that modifies global state.
- ☐ C Transforming a function with multiple arguments into a sequence of single-argument functions.
- ☐ D Applying a function to a list of arguments.

**Question 31 (L.O.2).** In *Assignment 1*, what does the Complex class represent for a DeathEater?

- Ⓐ The character's base HP.
- Ⓑ The character's mana as a complex number.
- Ⓒ The character's weapon power.
- Ⓓ The character's combat score.

**Question 32 (L.O.2).** What is the output of the following Python code?

```
class Shape:
 def area(self):
 return 0

class Rectangle(Shape):
 def __init__(self, width, height):
 self.width = width
 self.height = height
 def area(self):
 return self.width * self.height

shapes = [Shape(), Rectangle(3, 4)]
for shape in shapes:
 print(shape.area())
```

- Ⓐ 0  
12
- Ⓑ Compilation error
- Ⓒ 0  
0
- Ⓓ Runtime error

**Question 33 (L.O.3).** What is the output of the following Python code?

```
def is_even(x):
 return x % 2 == 0
numbers = [1, 2, 3, 4]
result = list(filter(is_even, numbers))
print(result)
```

- Ⓐ  $[1, 3]$ .                      Ⓑ  $[1, 2, 3, 4]$ .  
Ⓒ  $[2, 4]$ .                      Ⓓ  $[\ ]$ .

**Question 34 (L.O.1).** In Python, what keyword is used to call a base class constructor from a derived class?

- A** parent.   **B** super.   **C** base.   **D** extend.

**Question 35 (L.O.1).** In Java, why might you choose to use an interface over an abstract class?

- Ⓐ To store instance variables.
- Ⓑ To allow multiple inheritance of type.

- Ⓒ To prevent method overriding.
- Ⓓ To provide a complete implementation of methods.

**Question 36 (L.O.3).** Complete the following Python code to square numbers using a higher-order function:

```
numbers = [1, 2, 3]
result = list(_____)
print(result)
```

- Ⓐ filter(lambda x: x \* x, numbers)
- Ⓑ map(lambda x: x \* x, numbers)
- Ⓒ filter(lambda x: x > 0, numbers)
- Ⓓ map(lambda x: x + 1, numbers)

**Question 37 (L.O.2).** What is the output of the following C++ code?

```
#include <iostream>
class Base {
public:
 virtual void print() { std::cout << "
 Base\n"; }
};
class Derived : public Base {
public:
 void print() override { std::cout << "
 Derived\n"; }
};
int main() {
 Base* b = new Derived();
 b->print();
 delete b;
 return 0;
}
```

- Ⓐ Compilation error.
- Ⓑ Undefined behavior.
- Ⓒ Base.
- Ⓓ Derived.

**Question 38 (L.O.1).** What is a lambda function in Python?

- Ⓐ A named function defined with def.
- Ⓑ A function that modifies global state.
- Ⓒ A function with no return value.
- Ⓓ An anonymous function defined inline.

**Question 39 (L.O.1).** In Java, how is runtime polymorphism achieved?

- Ⓐ By using static methods.
- Ⓑ By overriding methods in a subclass and using a superclass reference.
- Ⓒ By defining private methods.
- Ⓓ By using final methods.

**Question 40 (L.O.1).** In Java, complete the following Observer pattern code to notify viewers:

```
interface StockViewer {
 void onUpdate(String price);
}
class StockFeeder {
 private List<StockViewer> viewers = new
 ArrayList<>();
 public void registerViewer(StockViewer
 viewer) {
 viewers.add(viewer);
 }
 public void notify(String price) {

 }
}
```

- Ⓐ for (StockViewer viewer : viewers) {  
viewer.onUpdate(price); }
- Ⓑ viewers.clear();
- Ⓒ return viewers;
- Ⓓ System.out.println(price);

**Question 41 (L.O.3).** In *Assignment 3*, what is the output of the following code for list\_of\_commands = ["INSERT x int", "LOOKUP x"]?

```
from StaticError import *
from Symbol import *
def simulate(list_of_commands):
 symbols = []
 def insert(name, typ):
 if any(s.name == name for s in
 symbols):
 raise Redeclared(f"INSERT {name}
 {typ}")
 symbols.append(Symbol(name, typ))
 return "success"
def lookup(name, index=len(symbols) - 1)
:
 if index < 0:
```



```

 raise Undeclared(f"LOOKUP {name}"
)
 elif symbols[index].name == name:
 return index
 else:
 return lookup(name, index - 1)
results = []
for cmd in list_of_commands:
 parts = cmd.split()
 if parts[0] == "INSERT":
 results.append(insert(parts[1],
 parts[2]))
 elif parts[0] == "LOOKUP":
 results.append(lookup(parts[1]))
return results

```

- Ⓐ ["error", 0]
- Ⓑ ["success", 0]
- Ⓒ []
- Ⓓ ["success", "error"]

**Question 42 (L.O.1).** What is polymorphism in OOP?

- Ⓐ The ability to hide data within a class.
- Ⓑ The ability to create multiple classes with the same name.
- Ⓒ The ability of different classes to be treated as instances of the same class through a common interface.
- Ⓓ The ability to inherit from multiple base classes.

**Question 43 (L.O.3).** In *Assignment 3*, what does the ASSIGN command do when the identifier is found?

- Ⓐ Updates the identifier's value in the symbol table.
- Ⓑ Raises a Redeclared error.
- Ⓒ Returns the block level.
- Ⓓ Updates the identifier's type.

**Question 44 (L.O.1).** Given the following Java code, which UML relationship best represents the connection between Person and Address?

```

class Address {
 String street;
}

```

```

class Person {
 private Address address;
 public Person(Address address) {
 this.address = address;
 }
}

```

- Ⓐ Inheritance.
- Ⓑ Dependency.
- Ⓒ Composition.
- Ⓓ Association.

**Question 45 (L.O.2).** What is the output of the following Java code?

```

interface Drawable {
 void draw();
}
class Circle implements Drawable {
 public void draw() { System.out.println(
 "Circle"); }
}
class Square implements Drawable {
 public void draw() { System.out.println(
 "Square"); }
}
public class Main {
 public static void main(String[] args) {
 Drawable[] shapes = {new Circle(),
 new Square()};
 for (Drawable shape : shapes) {
 shape.draw();
 }
 }
}

```

- Ⓐ Compilation error.
- Ⓑ Square\_Circle.
- Ⓒ Circle\_Square.
- Ⓓ Drawable.

**Question 46 (L.O.1).** In Python, which module provides the reduce higher-order function?

- Ⓐ math.
- Ⓑ sys.
- Ⓒ functools.
- Ⓓ os.

**Question 47 (L.O.1).** In the Singleton pattern in Java, why is the constructor typically made private?

- Ⓐ To allow multiple instances to be created.
- Ⓑ To prevent direct instantiation from outside the class.
- Ⓒ To make the class abstract.
- Ⓓ To enable inheritance.

**Question 48 (L.O.3).** Complete the following Python code to produce a list of cubes of odd numbers:

```
numbers = [1, 2, 3, 4, 5]
result = _____
print(result)
```

- (A)** `[x ** 3 for x in numbers]`
- (B)** `[x ** 3 for x in numbers if x % 2 != 0]`
- (C)** `[x for x in numbers if x % 2 != 0]`
- (D)** `[x * 3 for x in numbers if x % 2 != 0]`

**Question 49 (L.O.2).** What is the output of the following Python code?

```
class Animal:
 def __init__(self):
 self.name = "Animal"
 def show(self):
 print(self.name)
class Cat(Animal):
 def __init__(self):
 self.name = "Cat"
animal = Animal()
cat = Cat()
animal.show()
cat.show()
```

- (A)** `Cat_Animal`                      **(B)** `Cat_Cat`
- (C)** `Animal_Cat`                      **(D)** `Animal_Animal`

**Question 50 (L.O.1).** In C++, which of the following is true about a pure virtual function?

- (A)** It must be implemented in the base class.
- (B)** It cannot be overridden in derived classes.
- (C)** It makes the class abstract and must be overridden in derived classes.
- (D)** It is only used in interfaces.

**Question 51 (L.O.1).** In Python, what does the reduce function from the functools module do?

- (A)** Filters elements based on a condition.
- (B)** Combines elements in an iterable into a single value.
- (C)** Applies a function to each element in an iterable.

- (D)** Creates a new function.

**Question 52 (L.O.1).** In Java, what is the output of the following Observer pattern implementation?

```
interface StockViewer {
 void onUpdate(String price);
}
class StockFeeder {
 private List<StockViewer> viewers = new
 ArrayList<>();
 public void registerViewer(StockViewer
 viewer) {
 viewers.add(viewer);
 }
 public void notify(String price) {
 for (StockViewer viewer : viewers) {
 viewer.onUpdate(price);
 }
 }
}
class PriceView implements StockViewer {
 public void onUpdate(String price) {
 System.out.println("Price: " + price)
 }
}
public class Main {
 public static void main(String[] args) {
 StockFeeder feeder = new StockFeeder
 ();
 PriceView view = new PriceView();
 feeder.registerViewer(view);
 feeder.notify("200.0");
 }
}
```

- (A)** No output.                      **(B)** Compilation error.
- (C)** Price: 200.0.                      **(D)** Runtime error.

**Question 53 (L.O.3).** What is the output of the following Python code?

```
numbers = [1, 2, 3, 4]
doubled = list(map(lambda x: x * 2, numbers))
print(doubled)
```

- (A)** `[]`.                      **(B)** `[1, 4, 9, 16]`.
- (C)** `[1, 2, 3, 4]`.                      **(D)** `[2, 4, 6, 8]`.



**Question 54 (L.O.2).** In *Assignment 1*, what is the purpose of the TeamMaker class?

- ☐ A To calculate combat scores.
- ☐ B To create teams of combatants.
- ☐ C To manage game levels.
- ☐ D To store complex numbers.

**Question 55 (L.O.2).** In *Assignment 2*, what happens if a stock code is added to StockFeeder that is already being monitored?

- ☐ A A runtime exception is thrown.
- ☐ B The stock is added again.
- ☐ C The stock is ignored.
- ☐ D An error is logged to the console.

**Question 56 (L.O.3).** What is the output of the following Python code?

```
f = lambda x: lambda y: x - y
g = f(10)
print(g(4))
```

- ☐ A 4.
- ☐ B 14.
- ☐ C 6.
- ☐ D None.

**Question 57 (L.O.1).** In a UML class diagram, what does a dashed line with an open arrow indicate?

- ☐ A Composition.
- ☐ B Inheritance.
- ☐ C Dependency.
- ☐ D Association.

**Question 58 (L.O.1).** In Python, which of the following is a stateless function?

- ☐ A `def append(lst, x): lst.append(x); return lst`
- ☐ B `def add(a, b): return a + b`
- ☐ C `def increment(): global count; count += 1; return count`
- ☐ D `def log(x): print(x); return x`

**Question 59 (L.O.1).** In a UML class diagram, what does a solid line with a filled diamond indicate?

- ☐ A Dependency.
- ☐ B Association.
- ☐ C Composition.
- ☐ D Inheritance.

**Question 60 (L.O.2).** In *Assignment 2*, complete the following Java code to implement the Singleton pattern:

```
package com.myproject;
```

```
import java.util.*;
public class StockFeeder {
 private List<Stock> stockList = new
 ArrayList<>();
 private Map<String, List<StockViewer>>
 viewers = new HashMap<>();
 private static StockFeeder instance =
 null;
 private StockFeeder() {}
 public static StockFeeder getInstance()
 {

 }
}
```

- ☐ A `return new StockFeeder();`
- ☐ B `if (instance == null) instance = new StockFeeder(); return instance;`
- ☐ C `instance = new StockFeeder(); return instance;`
- ☐ D `return instance;`

**Question 61 (L.O.3).** What is the output of the following Python code?

```
from functools import reduce
numbers = [1, 2, 3, 4]
result = reduce(lambda x, y: x + y, numbers)
print(result)
```

- ☐ A 0.
- ☐ B 24.
- ☐ C 10.
- ☐ D 12.

**Question 62 (L.O.1).** In Java, which access modifier restricts a class member to be accessible only within the same class?

- ☐ A protected.
- ☐ B private.
- ☐ C public.
- ☐ D default.

**Question 63 (L.O.1).** In Python, what does the following curried function do when called as `f(5)(2)`?

```
f = lambda x: lambda y: x + y
```

- ☐ A Multiplies 5 and 2.
- ☐ B Raises an error.
- ☐ C Returns a lambda function.
- ☐ D Adds 5 and 2.

**Question 64 (L.O.3).** In *Assignment 3*, complete the

following Python code to handle the INSERT command:

```
from StaticError import *
from Symbol import *
def simulate(list_of_commands):
 symbols = []
 def insert(name, typ):

 def lookup(name, index=len(symbols) - 1):
 :
 if index < 0:
 raise Undeclared(f"LOOKUP {name}")
 elif symbols[index].name == name:
 return index
 else:
 return lookup(name, index - 1)
 results = []
 for cmd in list_of_commands:
 parts = cmd.split()
 if parts[0] == "INSERT":
 results.append(insert(parts[1],
 parts[2]))
 elif parts[0] == "LOOKUP":
 results.append(lookup(parts[1]))
 return results
```

- Ⓐ return "success"
- Ⓑ if any(s.name == name for s in symbols):  
    □ raise Redeclared(f"INSERT name typ")  
    symbols.append(Symbol(name, typ))  
    return "success"
- Ⓒ raise Undeclared(f"INSERT name typ")
- Ⓓ symbols.append(Symbol(name, typ))

**Question 65 (L.O.1).** In Java, analyze the following code and determine if it compiles:

```
interface Movable {
 void move();
}
abstract class Vehicle {
 abstract void start();
}
class Car extends Vehicle implements Movable {
```

```
public void move() { System.out.println(
 "Moving"); }
public void start() { System.out.println(
 "Starting"); }
}
public class Main {
 public static void main(String[] args) {
 Car car = new Car();
 car.move();
 car.start();
 }
}
```

- Ⓐ Fails to compile due to multiple inheritance.
- Ⓑ Fails to compile due to missing method implementations.
- Ⓒ Compiles successfully.
- Ⓓ Compiles but throws a runtime exception.

**Question 66 (L.O.3).** In *Assignment 3*, what is the purpose of the Symbol class?

- Ⓐ To manage block levels.
- Ⓑ To execute commands.
- Ⓒ To log errors.
- Ⓓ To store an identifier's name and type.

**Question 67 (L.O.1).** In Java, what is the benefit of encapsulation?

- Ⓐ It allows multiple inheritance.
- Ⓑ It prevents method overriding.
- Ⓒ It protects data by controlling access through methods.
- Ⓓ It enables runtime polymorphism.

**Question 68 (L.O.3).** What is the output of the following Python code?

```
from functools import reduce
def multiply(x, y):
 return x * y
numbers = [1, 2, 3, 4]
result = reduce(multiply, numbers)
print(result)
```

- Ⓐ 24.
- Ⓑ 10.
- Ⓒ 12.
- Ⓓ 0.

**Question 69 (L.O.2).** What is the output of the following Java code?

```

class Animal {
 String sound() { return "Generic"; }
}
class Dog extends Animal {
 String sound() { return "Woof"; }
}
public class Main {
 public static void main(String[] args) {
 Animal animal = new Dog();
 System.out.println(animal.sound());
 }
}

```

- Ⓐ Compilation error.      Ⓑ Runtime error.  
 Ⓒ Generic.      Ⓓ Woof.

**Question 70 (L.O.1).** In Java, which design pattern allows a subject to notify multiple observers of state changes?

- Ⓐ Factory.      Ⓑ Adapter.  
 Ⓒ Observer.      Ⓓ Singleton.

**Question 71 (L.O.1).** In Python, what does the following lambda function do?

```

f = lambda x: x + 5
print(f(3))

```

- Ⓐ Raises an error.  
 Ⓑ Multiplies the input by 5.  
 Ⓒ Adds 5 to the input.  
 Ⓓ Returns the input unchanged.

**Question 72 (L.O.2).** In *Assignment 1*, complete the following Java code to calculate the combat score:

```

public class Knight extends Fighter
 implements Combatable {
 private int baseHp;
 private int wp;
 public Knight(int baseHp, int wp) {

```

```

 this.baseHp = baseHp;
 this.wp = wp;
 }
 @Override
 public double getCombatScore() {

 }
}

```

- Ⓐ return baseHp \* wp;      Ⓑ return baseHp + wp;  
 Ⓒ return baseHp / wp;      Ⓓ return baseHp;

**Question 73 (L.O.3).** What is the output of the following Python code?

```

def add_n(n):
 return lambda x: x + n
numbers = [1, 2, 3]
result = list(map(add_n(3), numbers))
print(result)

```

- Ⓐ [3, 3, 3].      Ⓑ [4, 5, 6].  
 Ⓒ [1, 2, 3, 3].      Ⓓ [1, 2, 3].

**Question 74 (L.O.1).** Which of the following is an example of a curried function in Python?

- Ⓐ lambda x, y: x + y.  
 Ⓑ def add(x, y): return x + y.  
 Ⓒ lambda x: lambda y: x + y.  
 Ⓓ map(lambda x: x + 1, [1, 2, 3]).

**Question 75 (L.O.1).** In Python, if a class Dog inherits from a class Animal, and Animal has a method speak(), which statement is true?

- Ⓐ Dog cannot override the speak() method.  
 Ⓑ Dog must redefine speak() to use it.  
 Ⓒ Dog cannot access speak() unless it is private.  
 Ⓓ Dog automatically inherits speak() and can override it.

# KEYS

1. B 2. C 3. C 4. D 5. A 6. C 7. C 8. A 9. B 10. C 11. B 12. C 13. D 14. D 15. A  
16. C 17. D 18. C 19. C 20. B 21. B 22. C 23. A 24. A 25. A 26. C 27. A 28. B 29. C 30. C  
31. B 33. C 34. B 35. B 36. B 37. D 38. D 39. B 40. A 41. B 42. C 43. A 44. C 45. C 46. C  
47. B 48. B 49. C 50. C 51. B 52. C 53. D 54. B 55. D 56. C 57. C 58. B 59. C 60. B 61. C  
62. B 63. D 64. B 65. C 66. D 67. C 68. A 69. D 70. C 71. C 72. B 73. B 74. C 75. D

## SOLUTION GUIDE

### Question 1. Correct answer: **B**

Stateless programming means functions are pure—they produce the same output for the same input (deterministic) and have no side effects. Option a contradicts statelessness by modifying state. Option c violates purity by using global variables. Option d introduces side effects, which stateless programming avoids.

### Question 2. Correct answer: **C**

In Java, field access is resolved based on the reference type (static binding), not the actual object type. Here, `v` is of type `Vehicle`, so `v.display()` accesses the type field of `Vehicle`, outputting "Vehicle". Option b would require method overriding. Option a is incorrect as the code compiles. Option d is wrong as no null reference is involved.

### Question 3. Correct answer: **C**

The `map` function applies a given function to each element in an iterable, returning an iterator of the results. Option a describes `filter`. Option b describes `reduce`. Option d is unrelated to `map`.

### Question 4. Correct answer: **D**

The Singleton pattern ensures one instance by checking if instance is null and creating it only if necessary. Option d correctly implements this logic. Option a creates a new instance when one exists, breaking the pattern. Option b is invalid as it returns prematurely. Option c creates a new instance every time, breaking the pattern.

### Question 5. Correct answer: **A**

The `lambda f = lambda x: lambda y: x * y` creates a curried function. Calling `f(3)` returns

`lambda y: 3 * y`, which is assigned to `g`. Then `g(4)` computes `3 * 4 = 12`. Option b is incorrect (`3 + 4`). Option c is the input to `g`. Option d is incorrect as a value is returned.

### Question 6. Correct answer: **C**

Abstract classes can have instance variables and concrete methods, while interfaces (prior to Java 8) could not have instance variables or concrete methods (only constants and abstract methods). Java 8+ allows default methods in interfaces, but they still cannot have instance variables. Option a is incorrect as both can have concrete methods (post-Java 8 for interfaces). Option b is wrong as both can be inherited/implemented. Option d is incorrect as both can have private methods in modern Java.

### Question 7. Correct answer: **C**

In C++, runtime polymorphism is achieved by declaring methods as `virtual` in the base class, allowing derived classes to override them. This enables dynamic dispatch when calling methods through a base class pointer or reference. Option a is incorrect as static methods use compile-time binding. Option b is wrong as private methods are not inherited. Option d is unrelated to polymorphism.

### Question 8. Correct answer: **A**

The `filter` function with `lambda x: x % 2 == 0` selects even numbers from numbers, producing `[2, 4]`. Option b transforms elements into booleans. Option c filters odd numbers. Option d doubles all numbers.

### Question 9. Correct answer: **B**

The `DeathEater`'s combat score is the magnitude of its mana, computed as  $\sqrt{\text{real}^2 + \text{imaginary}^2}$ . For

real = 3 and imaginary = 4,  $\sqrt{3^2 + 4^2} = \sqrt{9 + 16} = 5.0$ . Option a is incorrect ( $3 * 4$ ). Option c is the sum ( $3 + 4$ ). Option d is the real part.

**Question 10.** Correct answer: **C**

The filter function is a higher-order function that takes a predicate function and an iterable, returning elements where the predicate is true. Option a (len) does not take a function. Option b (print) is not higher-order. Option d (int) is a type constructor.

**Question 11.** Correct answer: **B**

In Java, if a derived class constructor does not explicitly call a base class constructor using `super()`, the compiler automatically calls the base class's default (no-argument) constructor. If no default constructor exists, a compilation error occurs. Option c is only true if no default constructor exists. Option a is incorrect. Option d is false.

**Question 12.** Correct answer: **C**

The `fetch()` method retrieves stock data from `hoseLib`, converts each `HoseData` to a `StockPrice` using `convertToStockPrice`, and returns a list. Option c uses Java streams to map `HoseData` to `StockPrice` and collect the results. Option a returns an empty list. Option b returns `HoseData` directly, which is incompatible. Option d incorrectly maps `stockCodes`.

**Question 13.** Correct answer: **D**

The map function applies square to each element in numbers:  $1 * 1 = 1$ ,  $2 * 2 = 4$ ,  $3 * 3 = 9$ , producing `[1, 4, 9]`. Option a is empty. Option b doubles the numbers. Option c is the original list.

**Question 14.** Correct answer: **D**

The Singleton pattern ensures a class has only one instance and provides a global access point via a static `getInstance()` method. Option a (Observer) manages dependencies. Option b (Adapter) converts interfaces. Option c (Factory) is not specified.

**Question 15.** Correct answer: **A**

The list comprehension `[x + 1 for x in numbers]` adds 1 to each element in numbers:  $1 + 1 = 2$ ,  $2 + 1 = 3$ ,  $3 + 1 = 4$ ,  $4 + 1 = 5$ , producing `[2, 3, 4, 5]`. Option b is the original list. Option c adds 2 to odd indices. Option d subtracts 1.

**Question 16.** Correct answer: **C**

To initialize the base class `Vehicle`, the `Car` constructor must call the `Vehicle` constructor with the model parameter. Option c uses `super().__init__(model)` to invoke the base class constructor, setting `self.model`. Option a directly sets the attribute, bypassing the base constructor. Option b is incorrect as it does not pass `self`. Option d incorrectly sets a method.

**Question 17.** Correct answer: **D**

The `StockFeeder` maintains a Map of `StockViewer` objects, and viewers are registered via `registerViewer()`. The `StockViewer` objects exist independently, indicating an association. Option a is incorrect as the lifecycle is not tied. Option b is wrong as the relationship is persistent. Option c is incorrect as `StockViewer` is an interface.

**Question 18.** Correct answer: **C**

A pure function produces no side effects (e.g., does not modify external state) and returns the same output for the same input. Option a contradicts purity. Option b is against functional principles. Option d is incorrect as pure functions typically return values.

**Question 19.** Correct answer: **C**

The Adapter pattern converts the interface of a class into another interface that a client expects, enabling incompatible classes to work together. Option a relates to Observer. Option b relates to Singleton. Option d relates to Factory (not specified).

**Question 20.** Correct answer: **B**

The map function with `lambda x: x * 2` doubles each element in numbers, producing `[2, 4, 6, 8]`. Option a uses `filter`, which selects elements rather than transforming them. Option c filters positive numbers. Option d adds 2 instead of doubling.

**Question 21.** Correct answer: **B**

The `sound()` method is virtual, enabling polymorphism. A `Base*` pointing to a `Derived` object calls the overridden `sound()` in `Dog`, outputting "Woof". Option a would be correct without `virtual`. Option c is incorrect as there is output. Option d is wrong as the code compiles.



**Question 22.** Correct answer: **C**

A solid line with a closed arrowhead (triangle) indicates inheritance, pointing from the subclass to the superclass. Option a uses a plain solid line. Option b uses a dashed line. Option d uses an open diamond.

**Question 23.** Correct answer: **A**

Stateless programming means functions are pure: they produce the same output for the same input (deterministic) and have no side effects (e.g., modifying external state). Option b is incorrect as functions return values. Option c contradicts statelessness. Option d is irrelevant.

**Question 24.** Correct answer: **A**

In *Assignment 1*, a Warrior's combat score is `baseHp` if `wp == 1`, otherwise `baseHp / 10.0`. Option a correctly implements this logic (ignoring the square ground condition for simplicity). Option b adds `baseHp` and `wp`, which applies to Knight. Option c multiplies them, which is incorrect. Option d divides them, which is incorrect.

**Question 25.** Correct answer: **A**

The list comprehension `[x * 2 for x in numbers if x < 0]` selects negative numbers from `numbers` and doubles them. The negative numbers in `[-2, -1, 0, 1, 2]` are `-2` and `-1`, so `-2 * 2 = -4`, `-1 * 2 = -2`, resulting in `[-4, -2]`. Option b selects negatives without doubling. Option c is empty. Option d includes all numbers.

**Question 26.** Correct answer: **C**

In *Assignment 2*, the `PriceFetcher` interface defines a `fetch()` method to retrieve stock prices from external sources (e.g., `HosePriceFetchLib`), used by adapters like `HoseAdapter`. Option a relates to `StockViewer`. Option b is managed by `StockFeeder`. Option d is handled by `Logger`.

**Question 27.** Correct answer: **A**

Python achieves information hiding using name mangling with double underscores (e.g., `__attribute`), which alters the attribute name to make it harder to access directly from outside the class. Option b is incorrect as Python lacks a `private` keyword. Option c is wrong as `protected` is not a keyword. Option d defeats information hiding.

**Question 28.** Correct answer: **B**

The `LOOKUP` command searches for an identifier in the symbol table from the innermost to the outermost scope (highest to lowest index). Since loops are prohibited, we use recursion. Option b implements this by checking the base case (`index < 0`, meaning the identifier is not found), returning the current index if the name matches, or recursively calling `lookup` with a decremented index. Option a returns a dummy value. Option c raises the wrong error. Option d accesses only the first element's name and does not search.

**Question 29.** Correct answer: **C**

The Adapter pattern adapts `Adaptee`'s interface to match `Target`. The Adapter's `request()` method calls `specificRequest()` on the `Adaptee` and appends " via Adapter", outputting "Adaptee via Adapter". Option b is incorrect as it misses the adapter's addition. Option a is wrong as the code compiles. Option d is incorrect as no runtime error occurs.

**Question 30.** Correct answer: **C**

Currying transforms a function that takes multiple arguments (e.g., `f(x, y)`) into a sequence of functions, each taking one argument (e.g., `f(x)(y)`). Option a refers to function composition. Option b is unrelated to currying. Option d is incorrect as currying changes the function's structure.

**Question 31.** Correct answer: **B**

In *Assignment 1*, the `Complex` class represents the mana of a `DeathEater` as a complex number with real and imaginary parts, used to compute its combat score via magnitude. Option a relates to `Fighter` subclasses. Option c relates to `wp`. Option d is the result of the computation, not the class's purpose.

**Question 32.** The `Shape` class returns 0 for `area()`. The `Rectangle` class overrides `area()` to compute `width * height = 3 * 4 = 12`. The loop calls the appropriate `area()` method due to polymorphism, outputting `"0_12"`. Option b is wrong as Python does not have compilation errors here. Option c is incorrect as `Rectangle`'s area is not 0. Option d is incorrect as no runtime error occurs.



**Question 33.** Correct answer: **C**

The filter function applies `is_even` to numbers, keeping elements where the condition is true. The even numbers in `[1, 2, 3, 4]` are 2 and 4, so the output is `[2, 4]`. Option a selects odd numbers. Option b is the original list. Option d is empty.

**Question 34.** Correct answer: **B**

In Python, `super()` is used to call the base class constructor or methods from a derived class, e.g., `super().__init__()`. Option a is incorrect. Option c is not a Python keyword. Option d is used in Java (`extends`).

**Question 35.** Correct answer: **B**

Interfaces allow a class to implement multiple interfaces, enabling multiple inheritance of type, unlike abstract classes, which can only be extended singly. Option a is incorrect as interfaces cannot have instance variables. Option c is wrong as interfaces allow method implementation. Option d is incorrect as interfaces typically define abstract or default methods.

**Question 36.** Correct answer: **B**

The map function with `lambda x: x * x` squares each element in numbers, producing `[1, 4, 9]`. Option a uses `filter`, which selects elements rather than transforming them. Option c filters positive numbers. Option d adds 1 to each number.

**Question 37.** Correct answer: **D**

The `print()` method is virtual, enabling polymorphism. A `Base*` pointing to a `Derived` object calls the overridden `print()` in `Derived`, outputting "Derived". Option c would be correct without `virtual`. Option a is incorrect as the code compiles. Option b is wrong as the behavior is well-defined.

**Question 38.** Correct answer: **D**

A lambda function in Python is an anonymous (unnamed) function defined inline using the `lambda` keyword, e.g., `lambda x: x + 1`. Option a refers to regular functions. Option b is incorrect as lambda functions can be pure. Option c is wrong as lambda functions return values.

**Question 39.** Correct answer: **B**

Runtime polymorphism in Java is achieved through method overriding, where a subclass provides a specific implementation of a method defined in its superclass, and a superclass reference calls the overridden method. Option a is incorrect as static methods use compile-time binding. Option c is wrong as private methods are not inherited. Option d is incorrect as final methods cannot be overridden.

**Question 40.** Correct answer: **A**

The `notify()` method in the Observer pattern should iterate over all registered viewers and call their `onUpdate()` method with the price. Option a correctly implements this. Option b clears the list, which is incorrect. Option c is invalid as `notify()` is void. Option d only prints the price without notifying viewers.

**Question 41.** Correct answer: **B**

The `INSERT x int` command adds a `Symbol("x", "int")` to symbols at index 0, returning "success". The `LOOKUP x` command recursively searches from the highest index (0) and finds x at index 0, returning 0 (the block level). The result is `["success", 0]`. Option a is incorrect as the insert succeeds. Option c is wrong as there are results. Option d is incorrect as no error occurs.

**Question 42.** Correct answer: **C**

Polymorphism allows objects of different classes to be treated as objects of a common superclass or interface, typically through method overriding or interface implementation. Option a refers to encapsulation. Option b is incorrect. Option d refers to multiple inheritance.

**Question 43.** Correct answer: **A**

In *Assignment 3*, the `ASSIGN` command updates the value of an existing identifier in the symbol table if found. If not found, it raises an `Undeclared` error. Option b applies to `INSERT`. Option c applies to `LOOKUP`. Option d is incorrect as types are not updated.

**Question 44.** Correct answer: **C**

The Person class has a private Address address field, and the Address is passed to the Person constructor, indicating that the Address is a part of the Person and its lifecycle is tied to the Person. This is composition. Option a is incorrect as there is no extends. Option b would apply if Person only temporarily used Address. Option d would apply if Address were not tightly coupled.

**Question 45.** Correct answer: **C**

The Drawable array holds a Circle and a Square, both implementing Drawable. The draw() method is called on each, invoking the respective class's implementation, outputting "Circle \_Square" in the order of the array. Option b reverses the order. Option a is incorrect as the code compiles. Option d is wrong as "Drawable" is not printed.

**Question 46.** Correct answer: **C**

The functools module provides reduce, which is used for functional programming tasks like reducing an iterable to a single value. Option a (math) is for mathematical operations. Option b (sys) is for system parameters. Option d (os) is for system operations.

**Question 47.** Correct answer: **B**

The private constructor prevents external code from creating new instances, ensuring only one instance exists, controlled by getInstance(). Option a contradicts the Singleton goal. Option c is incorrect as Singleton classes are concrete. Option d is irrelevant.

**Question 48.** Correct answer: **B**

The list comprehension [x \*\* 3 for x in numbers if x % 2 != 0] selects odd numbers (1, 3, 5) and computes their cubes: 1 \*\* 3 = 1, 3 \*\* 3 = 27, 5 \*\* 3 = 125, producing [1, 27, 125]. Option a cubes all numbers. Option c selects odd numbers without cubing. Option d multiplies by 3 instead of cubing.

**Question 49.** Correct answer: **C**

The Animal class sets name to "Animal", and the Cat class overrides the constructor to set name to "Cat". animal.show() prints "Animal", and cat.show() prints "Cat", resulting in "Animal\_Cat". Options a, b, and d do not match the constructor behavior.

**Question 50.** Correct answer: **C**

A pure virtual function (declared with = 0) makes the class abstract, meaning it cannot be instantiated, and derived classes must override it. Option a is incorrect as pure virtual functions have no implementation in the base class. Option b is wrong as they must be overridden. Option d is incorrect as C++ does not have interfaces like Java.

**Question 51.** Correct answer: **B**

The reduce function applies a binary function to pairs of elements in an iterable, reducing them to a single value (e.g., summing a list). Option a describes filter. Option c describes map. Option d is unrelated.

**Question 52.** Correct answer: **C**

The StockFeeder notifies registered StockViewer objects of price updates. The PriceView prints "Price: 200.0" when notified. Option a is incorrect as there is output. Option b is wrong as the code compiles. Option d is incorrect as no runtime error occurs.

**Question 53.** Correct answer: **D**

The map function applies the lambda lambda x: x \* 2 to each element in numbers, doubling them: 1 \* 2 = 2, 2 \* 2 = 4, 3 \* 2 = 6, 4 \* 2 = 8, producing [2, 4, 6, 8]. Option a is empty. Option b squares the numbers. Option c is the original list.

**Question 54.** Correct answer: **B**

In BTL1, the TeamMaker class is responsible for creating teams by grouping combatants (e.g., Knight, Warrior) based on specific rules, such as balancing combat scores. Option a relates to the Combatable interface. Option c is not part of BTL1's functionality. Option d relates to the Complex class.

**Question 55.** Correct answer: **D**

According to Assignment 2's business rules, if a stock code is added to StockFeeder via addStock() and it already exists, an error message is logged to the console via the Logger class. Option a is incorrect as no exception is thrown. Option b is wrong as duplicates are not allowed. Option c is incorrect as the error is logged, not ignored.

**Question 56.** Correct answer: **C**

The lambda `f = lambda x: lambda y: x - y` creates a curried function. `f(10)` returns `lambda y: 10 - y`, which is assigned to `g`. Then `g(4)` computes  $10 - 4 = 6$ . Option a is the input to `g`. Option b is incorrect ( $10 + 4$ ). Option d is incorrect as a value is returned.

**Question 57.** Correct answer: **C**

A dashed line with an open arrow in UML indicates a dependency, meaning one class temporarily uses another. Option a uses a filled diamond. Option b uses a solid line with a closed arrow. Option d uses a plain solid line.

**Question 58.** Correct answer: **B**

A stateless (pure) function produces the same output for the same input and has no side effects. Option b (add) meets this criterion. Option a modifies the input list. Option c modifies a global variable. Option d produces a side effect (printing).

**Question 59.** Correct answer: **C**

A solid line with a filled diamond in UML indicates composition, where the part's lifecycle is tied to the whole (e.g., if the whole is destroyed, the part is too). Option a uses a dashed line. Option b is a plain solid line. Option d uses a closed arrowhead.

**Question 60.** Correct answer: **B**

The Singleton pattern ensures one instance by checking if instance is null and creating it only if necessary. Option b correctly implements this logic. Option a creates a new instance every time. Option c also creates a new instance each call. Option d may return null if instance is not initialized.

**Question 61.** Correct answer: **C**

The reduce function applies the lambda `lambda x, y: x + y` to pairs of elements in numbers:  $((1 + 2) + 3) + 4 = 10$ . Option a would require a zero initial value. Option b is multiplication. Option d is incorrect.

**Question 62.** Correct answer: **B**

The `private` access modifier restricts access to class members to within the same class, enforcing encapsulation. Option a (`protected`) allows access within

the same package and subclasses. Option c (`public`) allows access from anywhere. Option d (`default`, or `package-private`) allows access within the same package.

**Question 63.** Correct answer: **D**

The curried function `f` takes `x` and returns a lambda that takes `y` and returns `x + y`. `f(5)` returns `lambda y: 5 + y`, and `f(5)(2)` computes  $5 + 2 = 7$ . Option a would multiply. Option b is wrong as no error occurs. Option c is incorrect as the final call returns a value.

**Question 64.** Correct answer: **B**

The `INSERT` command checks if the identifier already exists; if so, it raises a `Redeclared` error. Otherwise, it adds the symbol and returns "success". Option b correctly implements this logic. Option a skips the check. Option c raises the wrong error. Option d skips the check and return value.

**Question 65.** Correct answer: **C**

The `Car` class implements the `Movable` interface by providing `move()` and extends the `Vehicle` abstract class by providing `start()`. Java allows a class to extend one class and implement multiple interfaces, so the code compiles and runs. Option a is wrong as this form of multiple inheritance is allowed. Option b is incorrect as all methods are implemented. Option d is incorrect as no runtime exception occurs.

**Question 66.** Correct answer: **D**

In **Assignment 3**, the `Symbol` class represents an identifier with its name and type, used to store entries in the symbol table. Option a is managed by the symbols list and block structure. Option b is the role of `simulate`. Option c is handled by `TestUtils`.

**Question 67.** Correct answer: **C**

Encapsulation protects data by making fields private and providing controlled access through getters and setters, improving security and maintainability. Option a relates to inheritance. Option b relates to final methods. Option d relates to polymorphism.

**Question 68.** Correct answer: **A**

The reduce function applies multiply to pairs of elements in numbers:  $((1 * 2) * 3) * 4 = 24$ . Option b is the sum. Option c is incorrect. Option d would require a zero initial value.

**Question 69.** Correct answer: **D**

In Java, method overriding enables runtime polymorphism. The `sound()` method in `Dog` overrides the one in `Animal`. When `animal.sound()` is called, Java uses dynamic dispatch to invoke `Dog::sound()` since the actual object is a `Dog`, outputting "Woof". Option c would be correct if there were no overriding. Option a is incorrect as the code compiles. Option b is incorrect as no runtime error occurs.

**Question 70.** Correct answer: **C**

The Observer pattern allows a subject to notify multiple observers of state changes, commonly used in event-driven systems. Option a (Factory) is not specified. Option b (Adapter) adapts interfaces. Option d (Singleton) ensures one instance.

**Question 71.** Correct answer: **C**

The lambda `lambda x: x + 5` adds 5 to its input. `f(3)` computes  $3 + 5 = 8$ . Option a is wrong as no error occurs. Option b would multiply. Option d is incorrect as the input is modified.

**Question 72.** Correct answer: **B**

In Assignment 1, a Knight's combat score is the sum of its `baseHp` and `wp`. Option b correctly returns

`baseHp + wp`. Option a multiplies them, which applies to other classes. Option c divides them, which is incorrect. Option d ignores `wp`.

**Question 73.** Correct answer: **B**

The `add_n(3)` function returns a lambda `lambda x: x + 3`. The `map` function applies this to numbers:  $1 + 3 = 4$ ,  $2 + 3 = 5$ ,  $3 + 3 = 6$ , producing `[4, 5, 6]`. Option a is incorrect. Option c includes an extra element. Option d is the original list.

**Question 74.** Correct answer: **C**

A curried function returns another function for each argument. Option c defines a lambda that takes `x` and returns a lambda that takes `y`, computing  $x + y$ . Option a takes both arguments at once. Option b is a regular function. Option d is a map operation.

**Question 75.** Correct answer: **D**

In Python, a derived class inherits all non-private methods from its base class and can override them. Option a is false as overriding is allowed. Option b is incorrect as redefinition is optional. Option c is wrong as Python does not use private methods in the same way as Java/C++.